Ajaia Docs — Architecture Document

ARCHITECTURE.md — Ajaia Docs Technical Design Document

This document outlines the architectural design, system boundaries, database schemas, authorization models, and technical trade-offs for Ajaia Docs.

Ø<ß¯ Architecture Focus

Ajaia Docs prioritizes the core document lifecycle:

$$\text{LOGIN} \longrightarrow \text{CREATE / UPLOAD} \longrightarrow \text{EDIT} \longrightarrow \text{SAVE} \longrightarrow \text{REOPEN} \longrightarrow \text{SHARE} \longrightarrow \text{AUTHORIZE (403)}$$

Rather than attempting to build a complex Google Docs replica with real-time WebSockets or enterprise OAuth within a short timebox, Ajaia Docs focuses on production-grade API design, strict backend access control, reliable persistence, and clean UX.

Ø<ß×þ System Overview

```
[ Frontend (React + Vite + TipTap) ]
%
%  REST API (JSON / Multipart)
%¼
[ Express.js Backend Middleware ]
% % %  Authentication (JWT Verification)
% % %  Authorization (verifyDocAccess: Owner / Editor)
%
%¼
[ Controllers & Models ]
% % %  User Controller
% % %  Document Controller
% % %  Share Controller
% % %  Upload Controller
%
%¼
[ MongoDB / Mongoose ]
% % %  User Collection
% % %  Document Collection
% % %  Share Collection
```

Ø>Ýà Technology Selection Rationale

Why TipTap Editor?

- **Headless & Unstyled**: Built on ProseMirror, providing complete UI control with Tailwind CSS without

bloat or unwanted iframe styles.

- **Structured Content Handling**: Supports HTML output (`editor.getHTML()`) and JSON document trees natively.
- **Extensible & Lightweight**: Allows modular addition of features (Bold, Headings, Lists, Underline) without bundling unused heavy components.

Why MongoDB & Mongoose?

- **Document-Native Storage**: HTML and structured document bodies fit naturally into MongoDB document models without requiring SQL table normalization for document content.
- **Flexible Schema Evolution**: Easily accommodates future document attributes (e.g. tags, version counters, folder references).
- **Relational Referencing**: Refers to `User` and `Document` via ObjectIds with unique compound indexes on the `Share` collection to enforce integrity.

Ø=ÜÊ Database Schema Design

1. User Model (`User`)

Stores team members and demo accounts.

- `_id`: ObjectId (Primary Key)
- `name`: String (Required, trimmed)
- `email`: String (Required, unique, lowercase)
- `passwordHash`: String (Hashed with `bcryptjs`, 10 rounds)
- `createdAt`: Date
- `updatedAt`: Date

2. Document Model (`Document`)

Stores core document state and ownership.

- `_id`: ObjectId (Primary Key)
- `title`: String (Default: "Untitled document")
- `content`: String (HTML content generated by TipTap)
- `owner`: ObjectId (Ref `User`, Indexed)
- `createdAt`: Date
- `updatedAt`: Date

Index: { owner: 1 } for rapid retrieval of user-owned documents.

3. Share Model (`Share`)

Stores access delegation and permission levels.

- `_id`: ObjectId (Primary Key)
- `document`: ObjectId (Ref `Document`, Required)
- `user`: ObjectId (Ref `User`, Required)
- `permission`: String (Enum: `['EDITOR']`, Default: `'EDITOR'`)
- `createdAt`: Date
- `updatedAt`: Date

Unique Compound Index: { document: 1, user: 1 } prevents duplicate share records for the same user on a given document.

Ø=Ý Authentication & Authorization Strategy

Authentication Flow

1. User logs in at /api/auth/login with email and password.
2. Server validates password against bcryptjs hash.
3. Server issues a signed JWT token containing { id, name, email }.
4. Frontend stores token in localStorage and attaches it to every request via Axios interceptors: Authorization:

Bearer <token>.

5. Backend authenticateToken middleware verifies token validity and populates req.user.

Authorization Engine (`verifyDocAccess`)

Backend security does not rely on frontend UI hiding. Authorization is strictly enforced at the middleware level:

- **Owner Access**: If `req.user.id === document.owner`, permission level is set to `OWNER` (full read, write, share, and delete rights).
- **Editor Access**: If user is not the owner, the middleware queries `Share.findOne({ document: docId, user: req.user.id })`. If found, permission level is set to `EDITOR` (read and write rights).
- **Forbidden Access (403)**: If no share record exists, the middleware aborts the request immediately and returns HTTP `403 Forbidden`:

```json
{ "error": "You don't have permission to access this document." }
```

Ø=ÜÁ File Import Workflow (.txt / .md)

1. Client submits file via POST /api/upload (multipart/form-data).
2. Multer middleware enforces a 5MB size limit and validates extension against .txt and .md.
3. Server reads raw text buffer and parses lines into structured HTML tags (<h1>, <h2>, <li>, <p>).
4. Server creates a new Document owned by req.user with the original file name as title.
5. Returns 201 Created payload; client redirects to the TipTap editor.

Ø=Þ€ Intentional Exclusions & Next Steps

What Was Intentionally Excluded:

- **Real-Time Collaboration**: Sockets/Yjs sync avoided in favor of reliable, single-writer HTTP debounced autosave.
- **Enterprise OAuth**: Demo authentication provided instant friction-free evaluator access without third-party key setup.
- **Version History**: Simple single-version state maintained to optimize database throughput.

What Would Be Built Next (Roadmap):

1. Granular View-Only Permissions: Add VIEWER permission level to Share schema.
2. Simple Document Revision History: Append snapshots to a Revisions collection on major edits.
3. In-Line Document Comments: Add comment threads tied to text selection offsets.